

OOP Project Report

Introduction

The program is a card game called "Durak".

To play this game, you need a standard deck of 36/52 cards. The cards should have numbers (2,3,4,5), 6, 7, 8, 9, 10, Jack, Queen, King and Ace, as well as four suits: HEARTS, DIAMONDS, CLUBS, and SPADES.

Game rules:

Before the game starts, the cards are shuffled randomly. Then, the trump suit is revealed. This is a special suit that can beat all other suit cards, but it cannot beat a higher-ranking card of the same suit.

Each player gets six cards. In our case, there are three players: one is a human player (us), and the other are computer opponents (bots).

The game begins. As a player, we start by choosing one card from our hand to discard. The opponent (the bot) must then respond with a stronger card, either a higher-ranking card or a card of the trump suit. If the bot cannot beat our card, they take it, and the round ends. Then the other bot makes move and so on.

If the bot manages to beat our card, it makes a move in the next round. After each round, all players check their hands. If a player has fewer than six cards left, they take a card from the deck until they have six cards again or until the deck runs out of cards.

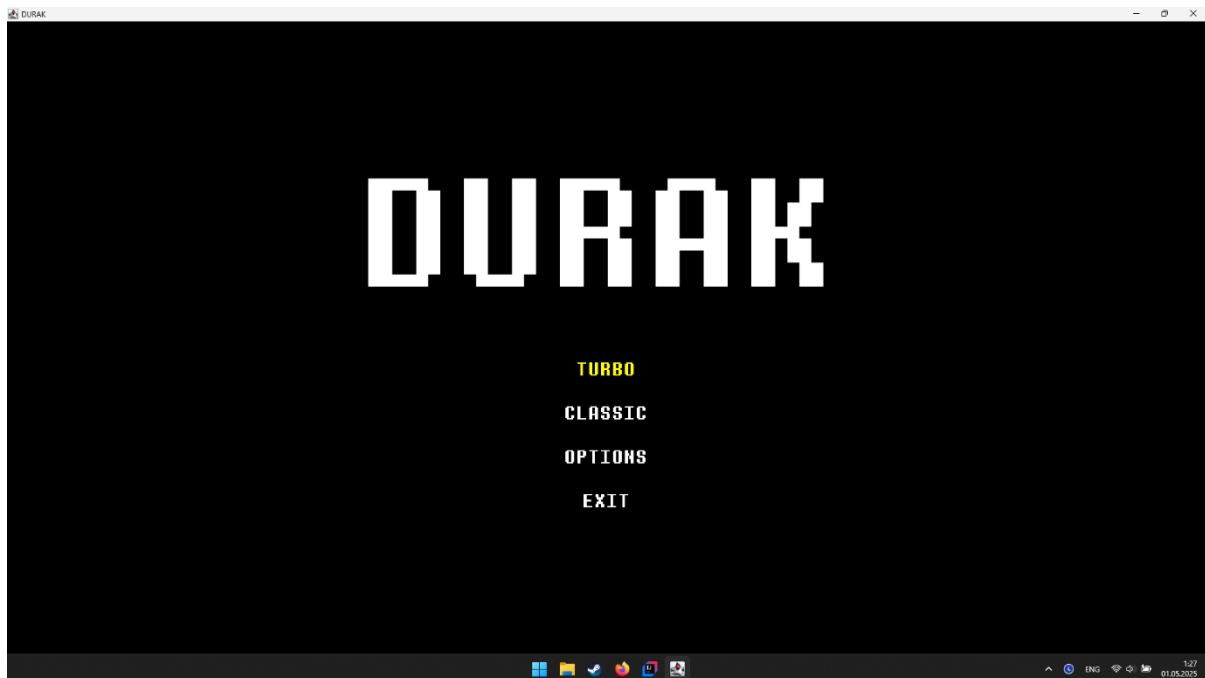
The game continues until one player has no cards left. That player wins the game.

Interaction and graphics:

The first thing I want to mention is using pixel game style, namely cards and font.

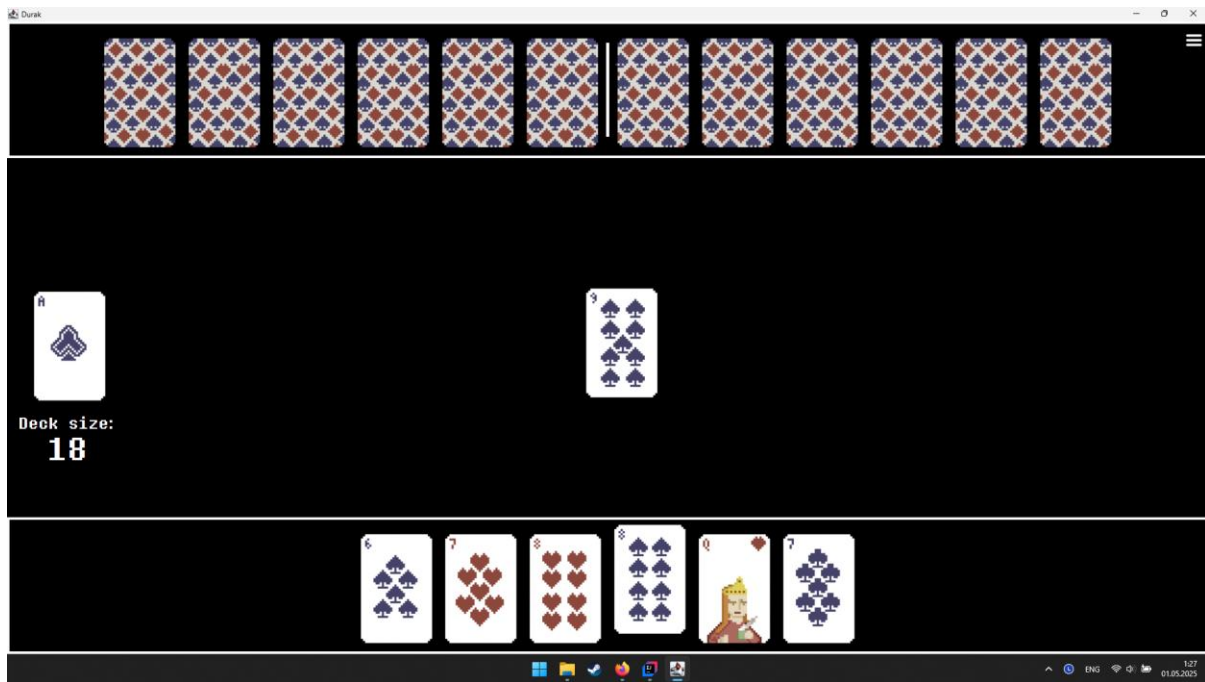


Now, let's discuss the interaction and graphics in the game. The graphics are built using JFrame Panels. When we launch the game, we are greeted with the main menu, which features buttons for choosing game mode and exiting it.

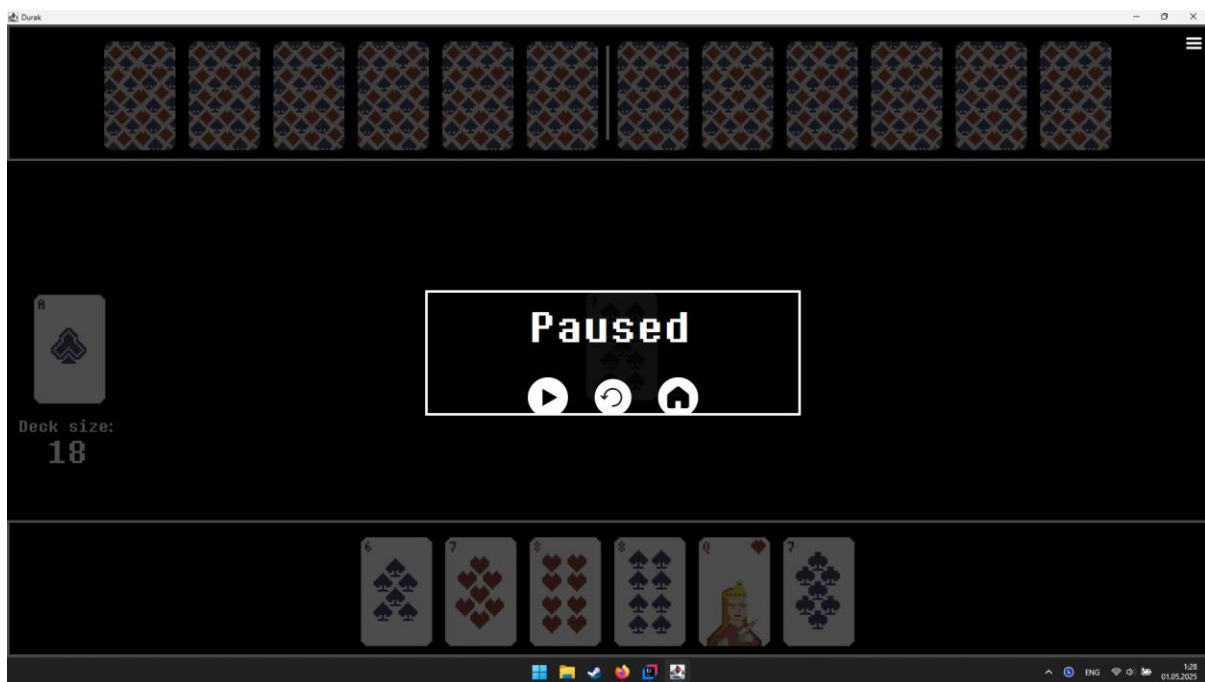


When we click on the "Turbo" button, we are taken to the fast game with 36 cards. But if we click on the "Classic" button, we are taken to the game with 52 cards.

Now you can see the game mode with 36 cards. Below we have cards that are in the player's hand. We can interact with these cards by clicking on them. On top of the screen, you'll see the cards of our opponents, which are not hidden from us. In the middle, there is a card that is hidden when it's our turn and shown when we must beat it. On the left, you'll see the number of cards remaining in the deck and the trump suit.



There is also pause button on top-right. After clicking on this button, we can see pause overlay with 3 options, resume restart and go to menu.



After completing the game, there is option to restart or exit.

OOP principles in this program:

Let's explore each of the object-oriented programming (OOP) principles along with exception handling, using examples from our Durak card game project:

1. Modularity

Modularity involves breaking down a software system into smaller, self-contained modules, each responsible for a specific functionality. This promotes code organization, reusability, and maintainability.

In the Durak game project, modularity is achieved by separating different aspects of the game into distinct classes. Each class handles specific functionalities, such as UI management, game logic, card handling, and player interactions.

Let's look at some of them. Card represents a single playing card with Rank and Suit. Deck generates an ArrayList of Card objects. GameParticipant (Abstract Class) - base class for all players (HumanPlayer, SimpleBot, LastBot), defines common behavior like playCard(). GameState stores game data (player hands, trump suit, current turn). ² GameUI manages the graphical interface.

GameSession controls the game flow (turns, win/lose conditions).

GameController coordinates between GameState, GameUI, and game logic.

GameLogicManager implements game rules (card comparisons, winning logic).

2. Hierarchy

Hierarchy refers to the arrangement of classes in a parent-child relationship, where child classes inherit attributes and behaviors from their parent classes. This facilitates code reuse and promotes polymorphism.

In the Durak game, hierarchy is implemented through:

Inheritance Structure:

GameParticipant<T> (Abstract Base Class)

- Defines core player logic (e.g., hand management, playCard() method).
- Uses generics (<T>) to support different move types.
- Child classes:
 - HumanPlayer (implements human move logic).
 - SimpleBot (basic AI for card attacks/defense).
 - LastBot (specialized AI for endgame scenarios).

Polymorphism in Action:

All child classes override playCard() with their own logic:

```
public class HumanPlayer extends GameParticipant<Integer> {
```

```
@Override 4 usages Zhalgas  
public void playCard(Integer card) {
```

Hierarchy provides:

- Reusability: Common code (e.g., drawCardFromDeck()) is shared via GameParticipant.
- Extensibility: New bot types can be added without modifying core logic.
- Polymorphism: The game calls playCard() without knowing the concrete player type.

3. Composition

Composition involves constructing complex objects by combining simpler ones. This promotes code reuse, flexibility, and modularity by allowing objects to be composed of other objects.

Composition is seen throughout the Durak game's architecture. Here are key examples:

GameUI Composition:

The GameUI class is composed of multiple Swing components to build the game interface:

- JPanel objects for different screen areas (player hand, bot hand, table)
- JLabel elements for displaying cards
- JButton for menu controls

This composition allows flexible UI arrangement without complex inheritance.

GameController Composition:

The GameController class combines several core components:

- GameState - manages game data like hands and trump suit
 - CardService - handles card comparison logic
 - GameLogicManager - implements game rules
 - SoundService - manages audio playback
- By composing these, the controller coordinates game flow while keeping responsibilities separated.

Deck Composition:

The Deck class is composed of:

- A List of Card objects representing the playing deck
 - Methods to shuffle and deal cards
- This encapsulates all deck-related functionality in one place.

Utility Composition:

The SoundService demonstrates composition by:

- Combining Clip objects for audio playback
- Managing multiple sound channels
- Providing a unified interface for sound effects

This compositional approach makes the system more modular - components can be modified or replaced independently without affecting the whole architecture. For example, changing the UI layout wouldn't require modifying game logic classes.

The extensive use of composition helps maintain clean separation of concerns throughout the Durak implementation.

4. Reuse

Reuse involves leveraging existing code or components for new purposes rather than reinventing them. This promotes efficiency, consistency, and maintainability by reducing redundancy.

The CardService interface and its implementation SimpleCardService contain reusable methods for card operations:

- `getCardRank()` – returns numeric value according to rank
- `getCardSuit()` – Returns card's suit
- `canBeat()` – check whether card can beat another card

```
@Override 5 usages  Zhalgas  
public String getCardSuit(String card) { return card.split(regex: "\\s+")[1]; }
```

The GameLogicManager provides reusable game rule methods:

- `sortHands()` – sorts cards in each hand
- `drawCardIfNeeded()` – draws card after each round if it's needed
- `checkWinner()` - Unified win-condition checking

Utility classes like ImageCache further promote reuse by centrally managing frequently accessed resources.

```
public class ImageCache { 4 usages  Zhalgas  
    private final Map<String, ImageIcon> cache = new HashMap<>(); 1 usage  
  
    public ImageIcon getImage(String path) { 1 usage  Zhalgas  
        return cache.computeIfAbsent(path, String p -> {  
            try {  
                return new ImageIcon(new File(p).getAbsolutePath());  
            } catch (Exception e) {  
                System.err.println("Error loading image: " + p);  
                return new ImageIcon();  
            }  
        })  
    }  
}
```

5. Encapsulation

Encapsulation is the bundling of data (fields) and methods (functions) that operate on the data into a single unit, typically a class. It ensures that the internal representation of an object is hidden from the outside and only necessary operations are exposed. Encapsulation allows us to control how data is accessed and modified within an object.

For example, in the Deck class, the list of cards is encapsulated within the class, and operations such as shuffling, drawing a card, and retrieving the deck size are provided through public methods:

```
public class Deck {  
  
    private final List<Card> cards; // Encapsulated data  
  
    public Deck(int mode) {  
  
        ...  
    }  
  
    public void shuffle() {  
        Collections.shuffle(cards);  
    }  
  
    public Card drawCard() {  
        return cards.isEmpty() ? null : cards.remove(0);  
    }  
  
    public int size() {  
        return cards.size();  
    }  
  
    public List<Card> getCards() {  
        return cards;  
    }  
}
```

Here, the cards list is encapsulated inside the Deck class, and direct access to it is restricted. Instead, controlled access is provided through methods like `shuffle()`, `drawCard()`, and `getCards()`.

6. Subtyping

Subtyping allows instances of a subclass to be treated as instances of their superclass, enabling polymorphism and dynamic method binding. In this project, subtyping is implemented through the abstract class `GameParticipant<T>` and its concrete subclasses: `HumanPlayer`, `SimpleBot`, and `LastBot`.

Inheritance

The `GameParticipant<T>` class serves as a generic base for all types of players (human or bot). It defines common behavior like managing the hand of cards, and declares the abstract method `playCard()` which must be implemented by each subclass:

```
public abstract class GameParticipant<T> {  
    protected List<String> hand = new ArrayList<>();  
    public abstract void playCard(T context);  
}
```

Concrete subclasses provide specific implementations:

```
public class HumanPlayer extends GameParticipant<Integer> {  
    @Override  
    public void playCard(Integer card) {  
        // Human player logic  
    }  
}
```

```
public class SimpleBot extends GameParticipant<Integer> {  
    @Override  
    public void playCard(Integer card) {  
        // Bot attack logic  
    }  
}
```

```

public class LastBot extends GameParticipant<Integer> {

    @Override

    public void playCard(Integer card) {

        // Bot defense logic

    }

}

```

Polymorphism and Dynamic Dispatch

Even though the subclasses are not always explicitly referenced through the superclass type, polymorphism is achieved when the game dynamically creates and uses different subclasses of `GameParticipant` based on the game state.

For example, in `GameSession.handleCardClick(int index)`:

```

case PLAYER -> {
    new HumanPlayer(gameState, this, gameUI,
    controller,cardService,gameLogicManager).playCard(null);
}
case BOT -> {
    botInstance.playCard(null);
}
case OPPONENT -> {
    new LastBot(gameState, this, gameUI,
    controller,cardService,gameLogicManager).playCard(null);
}

```

Here, although the references are typed as their specific classes, all of them override the same method `playCard(...)` declared in `GameParticipant`. This demonstrates polymorphic behavior: the game engine doesn't need to know the exact type—it simply calls the method, and the correct version is executed depending on the object's actual class.

Substitutability

This design allows the game logic (e.g., card turns) to treat all participants in a uniform way, using the same method call for different objects. The use of a common abstract base class promotes reusability, modularity, and extensibility, as new types of players or bots can be added by simply extending `GameParticipant` and implementing the required method.

7. Information Hiding

Information hiding is a fundamental object-oriented principle that involves restricting access to internal details of a class and exposing only what is necessary. This protects the integrity of the object's state, reduces interdependencies between components, and improves code maintainability.

In this project, the `GameState` class is a clear example of information hiding in practice.

Encapsulated Fields

All internal game data—such as player hands, deck, current turn, and trump suit—are declared as `private`. This means they cannot be accessed directly from outside the class:

```
public class GameState {  
    private List<String> playerHand = new ArrayList<>();  
    private List<String> botHand = new ArrayList<>();  
    private List<String> durakHand = new ArrayList<>();  
    private Deck deck;  
    private GameSession.Turn turn = GameSession.Turn.PLAYER;  
    private String trump;  
    ...  
}
```

Access Through Controlled Methods

The class provides public getter and setter methods to access and modify the internal state. This ensures that any access to the data goes through well-defined interfaces:

```
public List<String> getPlayerHand() {  
    return playerHand;  
}
```

This allows `GameState` to control how its data is accessed or modified, and to apply validation or constraints if needed.

Benefits of Information Hiding in `GameState`

- Other parts of the application cannot directly manipulate internal lists like `botHand` or `deck`. This prevents accidental or unauthorized changes.
- If the internal implementation changes (e.g., using a different collection type), the rest of the program does not need to be updated, as long as the public interface remains consistent.

- It promotes modularity and makes debugging and testing easier.

Relation to Encapsulation

- Encapsulation refers to bundling data and related behavior within a single class.
- Information Hiding is about restricting access to internal data using access modifiers (private, protected, public).

8. Abstraction

Abstraction is the process of hiding implementation details and exposing only the essential features of an object. In object-oriented programming, abstraction allows developers to work with high-level interfaces without concerning themselves with the internal workings of the components.

This concept is used extensively in the architecture of the Durak game implementation.

Abstract Class: `GameParticipant<T>`

The class `GameParticipant<T>` defines a high-level interface for all participants in the game (player or bots) without specifying how exactly each participant should behave:

```
public abstract class GameParticipant<T> {
    protected List<String> hand = new ArrayList<>();
    public abstract void playCard(T context);
}
```

This abstract class provides:

- A shared interface: all participants must implement `playCard(T context)`.
- Shared behavior: such as `drawCardFromDeck(...)` and `trimHand()`.

Subclasses like `HumanPlayer`, `SimpleBot`, and `LastBot` provide the concrete behavior:

```
public class SimpleBot extends GameParticipant<Integer> {
    @Override
    public void playCard(Integer context) {
        // Logic for bot attacking
    }
}
```

The game engine doesn't need to know how each participant plays — it only needs to know that every participant can play a card using `playCard()`. This is the essence of abstraction.

Interface: `CardService`

Another excellent example of abstraction is the `CardService` interface:

```
public interface CardService {  
    int getCardRank(String card);  
    String getCardSuit(String card);  
    boolean canBeat(String attackerCard, String defenderCard, String trumpSuit);  
    Comparator<String> createCardComparator(String trumpSuit);  
}
```

This interface defines what operations can be performed on cards (e.g., getting rank or checking if one card can beat another), but not how they are implemented.

The class `SimpleCardService` provides the actual implementation of the logic:

```
public class SimpleCardService implements CardService {  
    @Override  
    public int getCardRank(String card) {  
        // Parses rank from card string  
    }  
    ...  
}
```

By programming against the `CardService` interface, the game logic can remain flexible and decoupled from specific implementations. For example, you can later create an `AdvancedCardService` with different rules without changing the rest of the system.

Abstraction Benefits in This Codebase

- Promotes modularity: logic is cleanly separated across abstract components.
- Increases extensibility: new types of players, card logic, or UI can be added by extending base abstractions.
- Enhances readability and maintainability: developers can understand how components interact without diving into implementation details.

Abstraction allows this codebase to be scalable and adaptable — new behaviors can be added simply by implementing or extending existing abstract components.

9. Inheritance

Inheritance is a core concept in object-oriented programming that allows a class (subclass) to inherit fields and methods from another class (superclass). This promotes code reuse, abstraction, and extensibility.

In the Durak game project, inheritance is used to model different types of game participants and services.

Abstract Superclass

The abstract class `GameParticipant<T>` defines shared data (such as a hand of cards) and a contract for game participants:

```
public abstract class GameParticipant<T> {  
    protected List<String> hand = new ArrayList<>();  
    public abstract void playCard(T context);  
}
```

This class is extended by:

```
public class HumanPlayer extends GameParticipant<Integer> { ... }  
public class SimpleBot extends GameParticipant<Integer> { ... }  
public class LastBot extends GameParticipant<Integer> { ... }
```

Each subclass provides a specific implementation of the `playCard()` method, depending on whether it's a human player, an attacking bot, or a defending bot.

This use of inheritance creates an "is-a" relationship: each `HumanPlayer`, `SimpleBot`, and `LastBot` is a `GameParticipant`.

Sound Handling Inheritance

The class `SoundService` encapsulates all audio playback functionality. While not extended by other classes in this version of the code, it follows an inheritance-safe design with clear method responsibilities and encapsulated exception handling (see section 11).

10. Polymorphism

Polymorphism allows objects of different classes to be treated through a common interface or superclass, enabling flexibility, extensibility, and dynamic method resolution at runtime. The Durak game implements three major types of polymorphism: subtype (inheritance-based), parametric (generics), and method overloading.

Subtype Polymorphism

Subtype polymorphism is prominently used through the abstract class `GameParticipant<T>`, which acts as a common parent for both human and bots:

```
public abstract class GameParticipant<T> {  
    public abstract void playCard(T card);  
}
```

Two concrete subclasses implement this interface:

```
public class HumanPlayer extends GameParticipant<Integer> {  
    @Override  
    public void playCard(Integer card) { ... }  
}
```

```
public class SimpleBot extends GameParticipant<Integer> {  
    @Override  
    public void playCard(Integer card) { ... }  
}
```

In `GameSession`, the code dynamically determines the current turn and invokes the appropriate `playCard()` method without needing to know the exact class of the participant:

```
switch (gameState.getTurn()) {  
    case PLAYER -> new HumanPlayer(...).playCard(null);  
    case BOT -> botInstance.playCard(null);  
    case OPPONENT -> new LastBot(...).playCard(null);  
}
```

This promotes code reusability, extensibility, and clean abstraction.

Parametric Polymorphism (Generics)

Generics are used in the definition of `GameParticipant<T>`, enabling subclasses to specify the type they work with, while keeping the base class flexible and reusable:

```
public abstract class GameParticipant<T> {  
    public abstract void playCard(T card);  
}
```

This design allows future extensions of the game to operate on different data types without rewriting shared logic.

Method Overloading

Method overloading, which allows multiple methods with the same name but different parameters, is now implemented in the `SoundService` class:

```
public void playSoundEffect(String path) { ... }  
  
public void playSoundEffect(String path, boolean loop) {  
    try {  
        Clip clip = AudioSystem.getClip();  
        clip.open(AudioSystem.getAudioInputStream(new File(path)));  
        if (loop) {  
            clip.loop(Clip.LOOP_CONTINUOUSLY);  
        } else {  
            clip.start();  
        }  
    } catch (Exception e) {  
        showError("Sound error: " + e.getMessage());  
    }  
}
```

This allows flexible sound playback, where sound effects can either play once or loop continuously depending on the context. For example:

```
playSoundEffect(SoundTrack.GAME_THEME.path,true)  
  
playSoundEffect(SoundTrack.CARD_CLICK.path)
```

This use of overloading increases code clarity and eliminates the need for separate method names for similar actions.

11. Exception Handling

Exception handling is a critical component of robust software design. It enables the program to respond gracefully to unexpected conditions such as invalid input, missing files, or runtime resource issues. In the Durak game, exception handling is primarily implemented in file I/O, audio playback, and type parsing.

Audio Playback Exceptions

Most audio-related operations are enclosed in try-catch blocks to prevent the application from crashing if an audio file is missing, unsupported, or fails to load. This is implemented in the SoundService class:

```
public void playSoundEffect(String path) {  
    try {  
        Clip clip = AudioSystem.getClip();  
        clip.open(AudioSystem.getAudioInputStream(new File(path)));  
        clip.start();  
    } catch (Exception e) {  
        showError("Sound error: " + e.getMessage());  
    }  
}
```

An overloaded version of this method handles looping, but both versions gracefully handle any exceptions such as IOException, UnsupportedAudioFileException, or LineUnavailableException.

The showError() helper method displays an error dialog to inform the user without crashing the application:

```
private void showError(String message) {  
    JOptionPane.showMessageDialog(null, message, "ERROR", JOptionPane.ERROR_MESSAGE);  
}
```

This allows the game to continue running even if audio playback fails, ensuring a better user experience.

Type Parsing Exceptions

In the Main class, the deck mode is parsed from command-line arguments. A try-catch block is used to handle the potential NumberFormatException if the argument is not a valid integer:

```
int deckMode = 36;  
  
if (args.length > 0) {
```

```

try {
    deckMode = Integer.parseInt(args[0]);
} catch (NumberFormatException e) {
    deckMode = 36;
}
}

```

This ensures that the application has a valid fallback value and won't crash due to bad input.

Font Loading Fallback

In `GameStyle`, the game attempts to load a custom font from file. If the font file is not found or cannot be loaded, a fallback font is used:

```

public static Font loadGameFont(float size) {
    try {
        return Font.createFont(Font.TRUETYPE_FONT,
            new File("style/font/font.otf")).deriveFont(size);
    } catch (Exception e) {
        return new Font("Arial", Font.BOLD, (int) size);
    }
}

```

This pattern of graceful degradation ensures consistent UI behavior even when resources are missing or corrupted.

Exception handling is implemented consistently across the Durak game project to ensure fault tolerance and user-friendly error management. Key areas where exceptions are caught and handled include:

- Audio playback (`playSoundEffect`, `playBackgroundMusic`)
- File access (e.g., loading fonts and images)
- User input parsing (e.g., command-line arguments)

These safeguards contribute to a stable and reliable gaming experience, even in the presence of unexpected runtime issues.